

## LECTURE 4



# Computer Vision

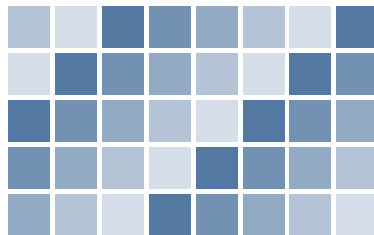
How computers see the world around us

With practical implementation

---

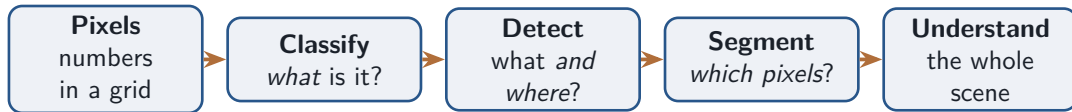
**Dr. Muhammad Kazim**

Department of Intelligent Systems  
University of Lahore



# Our journey today

---



We start with a single question: **how does a computer turn light into understanding?**

1

What is Computer Vision?

—

**Computer vision is teaching  
a  
computer to make sense of  
images.**

# You see a cat. The computer sees...

---



What **you** see  
“a cat”

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 190 | 30  | 90  | 150 | 210 | 50  |
| 90  | 150 | 210 | 50  | 110 | 170 |
| 210 | 50  | 110 | 170 | 230 | 70  |
| 110 | 170 | 230 | 70  | 130 | 190 |
| 230 | 70  | 130 | 190 | 30  | 90  |
| 130 | 190 | 30  | 90  | 150 | 210 |
| 30  | 90  | 150 | 210 | 50  | 110 |

What the **computer** sees  
a grid of numbers

# The “semantic gap”

---

There is a huge gap between **numbers** (what the computer has) and **meaning** (what we want).  
Computer vision is the bridge across that gap.



# Why is this hard?

---

The *same* object can produce **completely different** numbers.



**Lighting**

bright vs. dark



**Angle**

tilted, rotated



**Occlusion**

partly hidden



**Size / distance**

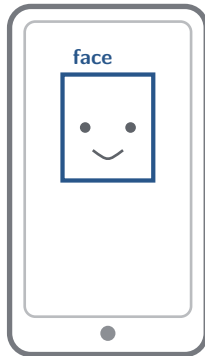
near vs. far

To us it's "obviously the same cat." To a computer, the numbers changed entirely.

# You already use it every day

---

- **Face unlock** on your phone
- **Self-driving** cars reading the road
- **Medical** scans flagging tumors
- **Cameras** that find and focus on faces
- **Search**: “find photos of the beach”



2

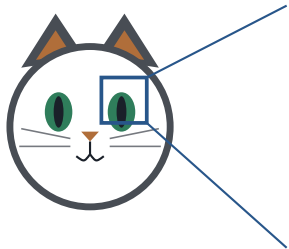
# How Images Are Represented

---

**To a computer, an image is  
just  
a big table of numbers.**

# Zoom in far enough... it's a grid of squares

---

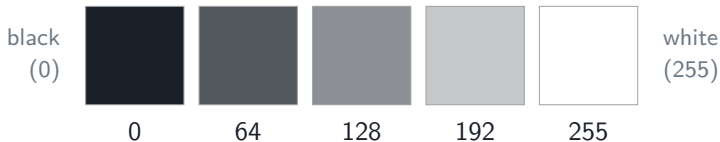


Each little square is a **pixel**.

# One pixel = one number (in grayscale)

---

A grayscale pixel's brightness is a single number, usually **0 to 255**.

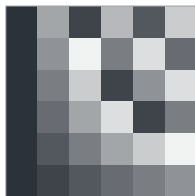


0 = black, 255 = white, everything between = a shade of gray.

# A grayscale image = a grid of those numbers

---

the picture



the same thing, as numbers

|    |     |     |     |     |     |
|----|-----|-----|-----|-----|-----|
| 20 | 152 | 42  | 174 | 64  | 196 |
| 20 | 130 | 240 | 108 | 218 | 86  |
| 20 | 108 | 196 | 42  | 130 | 218 |
| 20 | 86  | 152 | 218 | 42  | 108 |
| 20 | 64  | 108 | 152 | 196 | 240 |
| 20 | 42  | 64  | 86  | 108 | 130 |

A  $1000 \times 1000$  photo is a table of **a million** numbers.

# Color? Just use *three* numbers per pixel

---

Every color is a mix of **Red**, **Green**, and **Blue**: “RGB”.



$R = 220$



$G = 120$



$B = 60$

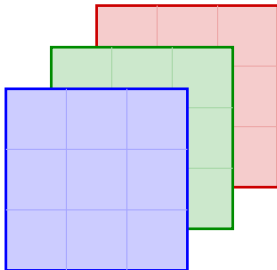


this color

Three numbers ( $R, G, B$ ) describe **any** color a pixel can take.

# So a color image is three grids, stacked

---



Height  $\times$  Width  $\times$  **3 channels**: a stack of numbers called a **tensor**.

# The key takeaway

---

Everything from here on is just **math on this grid of numbers**.  
The computer never “sees” a cat. It only computes with numbers.

▶ DEMO

# An image really is just numbers

We'll open a photo in code, zoom into a single pixel, and **print its (R, G, B) values** live, watching a picture turn into a grid of numbers.

*Interactive. Let's try it together.*

Code available at <https://github.com/Kazimbalti/intelligent-vision-systems>

3

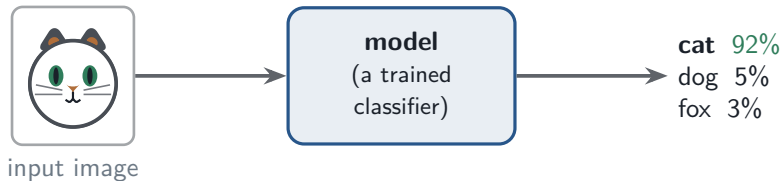
Image Classification

—

**Classification asks one  
question:  
“What is in this image?”**

# One image in, one label out

---

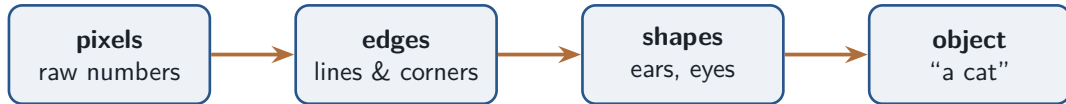


The output is a **probability** for each possible label.

# The trick: build up from simple to complex

---

A classifier learns to look for **patterns**, layer by layer.

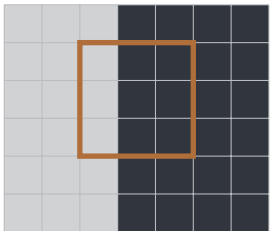


Early layers find edges; later layers combine them into meaningful parts.

# How does it “find an edge”?

---

Slide a tiny window over the image and look for a pattern.



the window sits on an edge (light → dark)

Where the window matches its pattern, it “lights up.”

Thousands of these detectors, learned automatically, make a **neural network**.

# Where do these detectors come from? Examples

---

We don't program the rules. We show the model a big **labeled dataset** (millions of pictures) and let it adjust itself.



“bird”



“dog”



“fox”



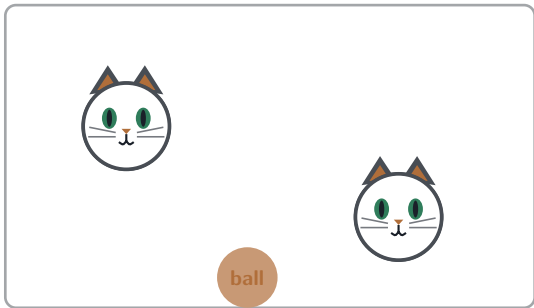
“cat”

guess → check →  
**adjust** → repeat  
(millions of times)

This is “**training**”: tuning the model on a labeled dataset (like ImageNet) until its guesses are right.

# One label isn't always enough

---



“This image is a cat”, but **which** cat?  
**where**? And what about the ball?

We need to know *where* things are. → **Detection.**

▶ DEMO

# Classify a photo

We'll feed an image to a trained classifier and read off its **top guesses with confidences**, then tweak the image and watch the predictions change.

*Interactive. Let's try it together.*

Code available at <https://github.com/Kazimbalti/intelligent-vision-systems>

4

Object Detection

**Detection asks:**

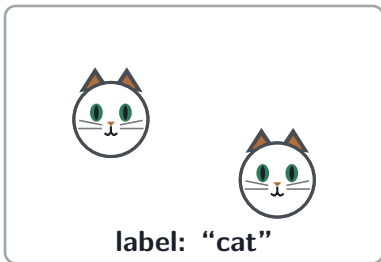
**“What objects are here,  
and where?”**

# Classification vs. Detection

---

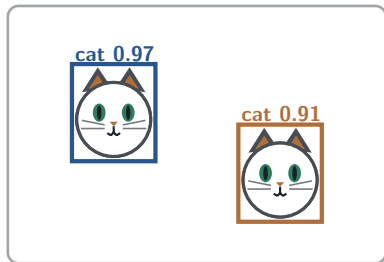
## Classification

one label for the whole image



## Detection

a box + label for each object

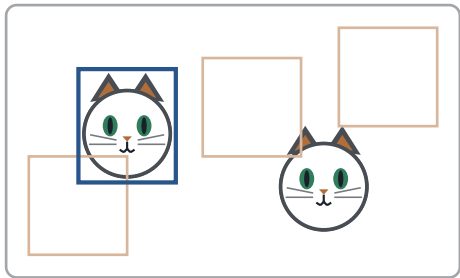


A “**bounding box**” is just four numbers: x, y, width, height.

# The naive idea: look everywhere

---

Slide a box of many sizes across the image and **classify each crop**.

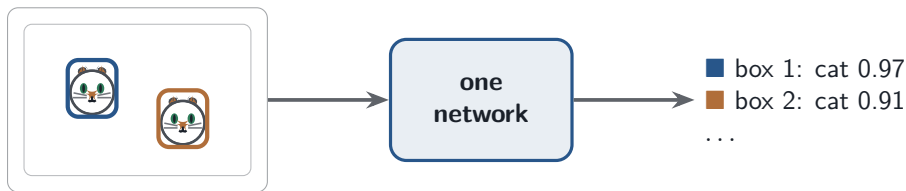


Works, but **slow**: millions of crops per image. Modern methods are cleverer.

# Modern idea: predict all boxes at once

---

The network looks at the **whole image once** and directly outputs a list of boxes + labels + confidences.



Fast enough to run on live video (this is how “real-time” detection works).

# Two practical wrinkles

## 1. Confidence threshold

Each box has a score. Keep only confident ones (e.g.  $> 0.5$ ), drop the rest.



cat 0.97 ✓



? 0.12 ✗

## 2. Too many boxes

The model may fire several boxes on **one** object. Merge the overlapping ones, keep the best.



A box tells us *where*, roughly, but not the object's exact shape. → **Segmentation.**

▶ DEMO

# Detect objects

We'll run a detector on a scene and watch **labelled boxes appear** around each object it finds.

*Interactive. Let's try it together.*

Code available at <https://github.com/Kazimbalti/intelligent-vision-systems>

5

# Object Segmentation

---

**Segmentation asks:**

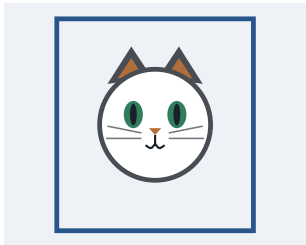
**“Which exact pixels  
belong to each object?”**

# From a box to a **mask**

---

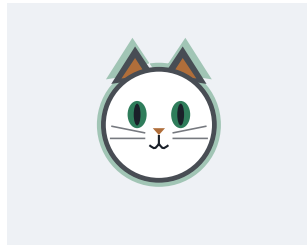
## Detection: a box

includes background corners



## Segmentation: a mask

exactly the cat's pixels

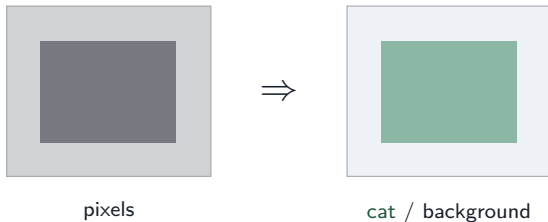


A mask labels **every pixel**: "cat" or "not cat."

# The idea: classify *every single pixel*

---

Instead of one label per image, produce one label **per pixel**.

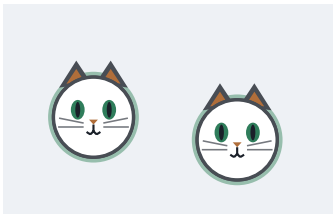


# Two flavors of segmentation

---

## Semantic

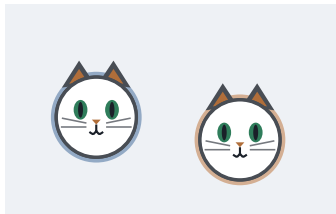
“which pixels are *a* cat?”



both cats share **one** color

## Instance

“which pixels are *this* cat?”



each cat gets its **own** color

Instance segmentation = detection *and* a per-object mask.

# Why go to all this trouble?

---

- **Medical imaging**: outline a tumor's exact boundary
- **Photo editing**: cut out the subject, blur the background
- **Self-driving**: know precisely where the road ends
- **Robotics**: grasp an object at the right spot

The more precisely we label pixels, the more a machine can **act** on what it sees.

▶ DEMO

# Paint the pixels

We'll run segmentation on an image, then use the mask to **blur or replace the background**, keeping only the subject's pixels.

*Interactive. Let's try it together.*

Code available at <https://github.com/Kazimbalti/intelligent-vision-systems>

# 6

## Image Understanding

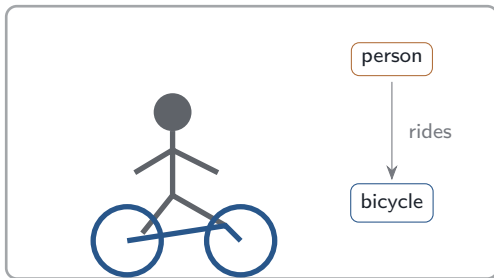
---

**Understanding asks:**

**“What is going on  
in this scene?”**

## Beyond “what” and “where”: relationships

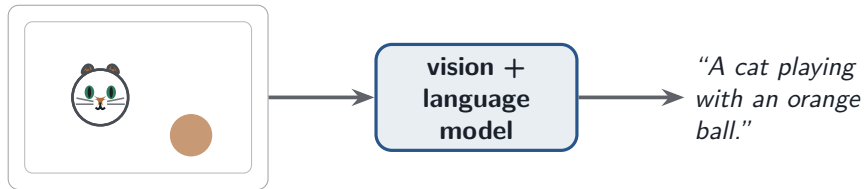
---



Not just “person” + “bicycle,” but **“a person is riding a bicycle.”**

# From pixels to a sentence

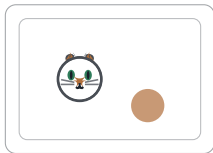
---



Modern models connect vision to **language**: describe images, answer questions about them.

# You can even ask questions

---



**Q:** "How many objects are there?"

**A:** "Two: a cat and a ball."

**Q:** "What color is the ball?"

**A:** "Orange."

This is called **Visual Question Answering (VQA)**.

# Understanding is still the frontier

---

- **Context:** is that a real fire, or a picture of one?
- **Intent:** is the person waving hello or hailing a cab?
- **Common sense:** cups hold liquid; glass breaks; shadows aren't objects

Computers are getting remarkably good at **seeing**. Truly **understanding**, like we do, is the hard, open problem.

▶ DEMO

# Ask the image a question

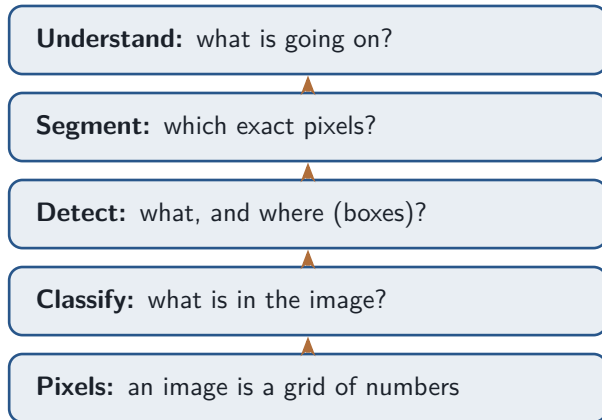
We'll run an image through a vision-language model and **ask it questions about the scene**, then see how well it understands what is going on.

*Interactive. Let's try it together.*

Code available at <https://github.com/Kazimbalti/intelligent-vision-systems>

# The ladder we climbed

---



**Every step is just  
smarter math on a grid of  
numbers.**

# Thank you

Questions?